Studio Nx

Backend(FastAPI/Python) 08 — User 모델 컬럼

2026-10-07

Backend(FastAPI/Python) 08 — `User` 모델 컬럼

항목	내용
파일	`studionx/backend/app/models/user.py`
작업	P1-03 백엔드 공통 — ORM 모델 컬럼 · `docs/DB.md` §3.1 · `studionx/db/01_phase1_account.sql`
작성	Claude (AI) · 2026-10-06 · 사용자 검토
이전 · 다음	07 `get_current_user_optional()` · 09 `Inquiry` 모델

1. 하는 일

MySQL 의 `users` 테이블을 **파이썬 클래스** `User` 로 옮겨 적는다. 그러면 SQL 문 대신 파이썬 코드로 회원을 읽고 쓸 수 있다.

SQL	파이썬 (이 클래스가 있으면)
`SELECT * FROM users WHERE user_id = 7`	`db.get(User, 7)`
`user.status` 칸 읽기	`user.status` → `UserStatus.ACTIVE`
`INSERT INTO users (email, …) VALUES (…)`	`db.add(User(email=…, …))`

테이블은 이 클래스가 만들지 않는다. 테이블은 `studionx/db/01_phase1_account.sql` 로 이미 만들어져 있고, 이 클래스는 그 모양을 그대로 따라 적은 것이다. 지금까지는 `user_id` 한 칸만 있었고, 이번에 나머지 14칸을 채웠다.

2. 언제 쓰나

회원을 다루는 모든 백엔드 코드가 이 클래스를 쓴다.

쓰는 곳	읽거나 쓰는 칸
06 `get_current_user()`	`status` — `ACTIVE` 인지
P1-04 회원가입	`email` · `password_hash` · `nickname` · `privacy_agreed_at` · `privacy_version` 넣기
P1-05 로그인	`email` 로 찾기 · `password_hash` 비교 · `last_login_at` 기록
P1-08 회원 정보 (userInfo.html)	`nickname` · `profile_image_path` 수정, 탈퇴 시 `status = DELETED` · `deleted_at`
P1-16 대시보드 (myPage.html)	`storage_quota_bytes` · `storage_used_bytes` — 저장 공간 막대
P1-21 관리자 화면	`role` — `ADMIN` 만 들어감

3. 코드

import (파일 맨 위)

```
from datetime import datetime
from enum import StrEnum

from sqlalchemy import BigInteger, Enum, FetchedValue, String, text
```

```
from sqlalchemy.dialects.mysql import DATETIME
from sqlalchemy.orm import Mapped, mapped_column

from app.db.session import Base
```

`BigInteger` · `Mapped` · `mapped_column` · `Base` 는 원래 있었고, 나머지를 이번에 추가했다. 모두 이 클래스에서 쓴다.

import	가져오는 것	쓰는 곳
`datetime` (파이썬 기본)	날짜 · 시각 타입	`created_at` 등 시각 칸
`StrEnum` (파이썬 기본)	값이 글자인 열거형	`UserRole` · `UserStatus`
`BigInteger` · `String`	SQL 의 `BIGINT` · `VARCHAR(n)`	숫자 · 글자 칸
`Enum`	SQL 의 `ENUM(…)`	`role` · `status`
`text`	SQL 글자를 그대로 넘김	기본값 `CURRENT_TIMESTAMP(3)` 등
`FetchedValue`	"DB 가 값을 바꾼다" 표시	`updated_at`
`DATETIME` (MySQL 전용)	`DATETIME(3)` — 밀리초까지	시각 칸
`Mapped` · `mapped_column`	칸 선언	모든 칸
`Base`	모든 모델의 부모 (`app/db/session.py`)	`class User(Base)`

함수 (이 작업은 함수가 아니라 클래스 · 칸 선언이다)

```
class UserRole(StrEnum):
    USER = "USER"
    ADMIN = "ADMIN"


class UserStatus(StrEnum):
    ACTIVE = "ACTIVE"
    SUSPENDED = "SUSPENDED"
    DELETED = "DELETED"


class User(Base):
    __tablename__ = "users"

    user_id: Mapped[int] = mapped_column(BigInteger, primary_key=True, autoincrement=True)

    email: Mapped[str] = mapped_column(String(255), unique=True)
    password_hash: Mapped[str] = mapped_column(String(255))
    nickname: Mapped[str] = mapped_column(String(50), unique=True)
    profile_image_path: Mapped[str | None] = mapped_column(String(500))
    role: Mapped[UserRole] = mapped_column(Enum(UserRole), server_default=UserRole.USER)
    status: Mapped[UserStatus] = mapped_column(Enum(UserStatus), server_default=UserStatus.ACTIVE)
    last_login_at: Mapped[datetime | None] = mapped_column(DATETIME(fsp=3))
    privacy_agreed_at: Mapped[datetime] = mapped_column(DATETIME(fsp=3))
    privacy_version: Mapped[str] = mapped_column(String(20))
    storage_quota_bytes: Mapped[int] = mapped_column(BigInteger, server_default=text("5368709120"))
    storage_used_bytes: Mapped[int] = mapped_column(BigInteger, server_default=text("0"))
    created_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)")
    )
    updated_at: Mapped[datetime] = mapped_column(
```

```
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)"), server_onupdate=FetchedValue()
    )
    deleted_at: Mapped[datetime | None] = mapped_column(DATETIME(fsp=3))
```

SQL 과 한 줄씩 비교

SQL (01_phase1_account.sql)	파이썬
`email VARCHAR(255) NOT NULL` + `UNIQUE KEY uq_users_email`	`Mapped[str]` · `String(255)` · `unique=True`
`profile_image_path VARCHAR(500) NULL`	`Mapped[str | None]`
`role ENUM('USER','ADMIN') NOT NULL DEFAULT 'USER'`	`Enum(UserRole)` · `server_default=UserRole.USER`
`last_login_at DATETIME(3) NULL`	`Mapped[datetime | None]` · `DATETIME(fsp=3)`
`storage_quota_bytes BIGINT UNSIGNED NOT NULL DEFAULT 5368709120`	`BigInteger` · `server_default=text("5368709120")` (5GB)
`updated_at … DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3)`	`server_default=text(…)` · `server_onupdate=FetchedValue()`

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름` 만 가져와 그대로 씀
`from sqlalchemy.dialects.mysql import …`	SQLAlchemy 안의 **MySQL 전용** 타입

식	뜻
`Mapped[str]`	이 칸의 파이썬 타입. **`None` 이 없으면 `NOT NULL`**, `Mapped[str | None]` 이면 `NULL` 허용
`mapped_column(String(255))`	SQL 타입 `VARCHAR(255)`
`unique=True`	같은 값을 두 번 넣을 수 없음 (SQL 의 `UNIQUE KEY`)
`class UserStatus(StrEnum)`	정해진 값만 쓰게 하는 목록. `UserStatus.ACTIVE == "ACTIVE"` 가 참
`server_default=…`	값을 안 넣으면 **DB 가** 채우는 기본값. 파이썬은 넣지 않고 DB 에 맡김
`text("CURRENT_TIMESTAMP(3)")`	글자 `'CURRENT_TIMESTAMP(3)'` 가 아니라 SQL 식으로 넘김
`server_onupdate=FetchedValue()`	"행을 고치면 DB 가 이 칸을 바꾼다"는 표시. 저장 뒤 새 값을 다시 읽어 오게 함
`DATETIME(fsp=3)`	`DATETIME(3)`. `fsp` = 초 아래 자릿수 (3 = 밀리초)

4. 흐름 (회원 하나를 넣을 때)

1. `User(email=…, password_hash=…, nickname=…, privacy_agreed_at=…, privacy_version=…)` 를 만든다. 다른 칸은 비워 둔다.
2. `db.add(user)` → `db.flush()` 로 DB 에 `INSERT` 한다.
3. DB 가 `user_id` · `role=USER` · `status=ACTIVE` · `storage_quota_bytes=5GB` · `created_at` 을 채운다.
4. `db.refresh(user)` 로 채워진 값을 다시 읽으면 `user.status` 는 `UserStatus.ACTIVE` 이다.

5. 왜 이렇게

선택	이유
기본값은 `server_default` (DB 가 채움)	기본값의 원본은 SQL 파일 하나다. 파이썬에도 따로 적으면 둘이 달라질 수 있다. SQLAdmin 이나 DB 도구로 직접 넣어도 같은 값이 들어간다
`role` · `status` 를 `StrEnum` 으로	오타(`"ACTIVEE"`)를 막고, 편집기가 값 목록을 보여 준다. 06 의 `user.status != "ACTIVE"` 도 그대로 동작한다
시각 칸은 `DATETIME(3)` (시간대 없음)	DB 에는 UTC 로 저장하기로 했다 (`docs/DB.md`). 읽은 값에는 시간대 정보가 없으니 UTC 로 보고 쓴다
`BIGINT UNSIGNED` 를 `BigInteger` 로	테이블은 SQL 파일이 만들므로 부호 여부는 DB 가 지킨다. 파이썬에서 읽고 쓰는 데는 차이가 없고, 다른 모델과도 같다
관계(`relationship`)는 아직 안 넣음	`user.projects` 처럼 쓸 곳이 생길 때(P1-09 등) 넣는다. 지금 넣으면 쓰지 않는 코드가 된다

6. 시험 (`tests/test_user_model.py`, 새로 만듦 · 4개)

실제 MySQL 의 `users` 테이블과 비교했다. DB 에 연결할 수 없으면 건너뛴다. 넣은 행은 시험이 끝나면 되돌려서 (rollback) DB 에 남지 않는다 (시험 뒤 확인: 0행).

시험	확인한 것
칸이 DB 와 같은지	칸 이름 15개 · 순서 · `NULL` 허용 여부가 실제 테이블과 같음
중복 금지 칸	`email` · `nickname` 두 칸
넣으면 DB 가 기본값을 채우는지	5칸만 넣고 저장 → `role` `USER` · `status` `ACTIVE` · 저장 공간 5GB · 사용량 0 · `created_at` 채워짐 · `last_login_at` · `deleted_at` 비어 있음
06 `get_current_user()` 를 실제 DB 로	`ACTIVE` 회원 → 그 회원, `DELETED` 회원 → 401

06 · 07 시험은 가짜 DB 세션으로 했는데, 이번에 칸이 생겨서 실제 DB 로도 확인했다.

전체 시험 52개 통과, `ruff` 검사 통과.